AIAC Assignment – 16.2

K.Lokesh	2303A51201	B21

Task Descrption-1: (Design Database Schema for Student Management System)

Instructions:

- Using AI tools, generate CREATE TABLE statements for a Student Management Database.
- Tables: Students, Courses, Enrollments.
- Include primary keys, foreign keys, and suitable data types.
  Ensure the schema follows normalization rules (1NF, 2NF, 3NF).

```sql
CREATE TABLE Students (
    student_id INTEGER PRIMARY KEY AUTOINCREMENT,
    first_name TEXT NOT NULL,
    last_name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL,
    date_of_birth DATE NOT NULL
);
CREATE TABLE Courses (
    course_id INTEGER PRIMARY KEY AUTOINCREMENT,
    course_name TEXT NOT NULL,
    course_code TEXT UNIQUE NOT NULL,
    credits INTEGER NOT NULL
);
CREATE TABLE Enrollments (
    enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
    student_id INTEGER NOT NULL,
    course_id INTEGER NOT NULL,
    enrollment_date DATE NOT NULL,
    FOREIGN KEY (student_id) REFERENCES Students(s
    FOREIGN KEY (course_id) REFERENCES Courses(cou
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE Students (
student_id INTEGER PRIMARY KEY AUTOINCREMENT,
first_name TEXT NOT NULL,
last_name TEXT NOT NULL,
email TEXT UNIQUE NOT NULL,
date_of_birth DATE NOT NULL
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE Courses (
course_id INTEGER PRIMARY KEY AUTOINCREMENT,
course_name TEXT NOT NULL,
course_code TEXT UNIQUE NOT NULL,
credits INTEGER NOT NULL
);
```

SQL ▾ 1 / 1 1 - 0 of 0

```sql
CREATE TABLE Enrollments (
enrollment_id INTEGER PRIMARY KEY AUTOINCREMENT,
student_id INTEGER NOT NULL,
course_id INTEGER NOT NULL,
enrollment_date DATE NOT NULL,
FOREIGN KEY (student_id) REFERENCES
Students(student_id),
FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

Task Description- 2: (Insert Sample Records)

Instructions:

- Use AI tools to generate INSERT statements to populate the tables.
- Insert at least 3 students, 4 courses, and 5 enrollment records.
  Verify data using SELECT queries.

Sample Code:

INSERT INTO Students VALUES

(1, 'Arjun', 'arjun@gmail.com'),

(2, 'Meera', 'meera@gmail.com'),

(3, 'Rahul', 'rahul@gmail.com');

INSERT INTO Courses VALUES

(101, 'Database Systems', 4),

(102, 'Data Structures', 3),

(103, 'Operating Systems', 4),

(104, 'AI Fundamentals', 3);

INSERT INTO Enrollments VALUES

(1, 1, 101),

(2, 1, 102),

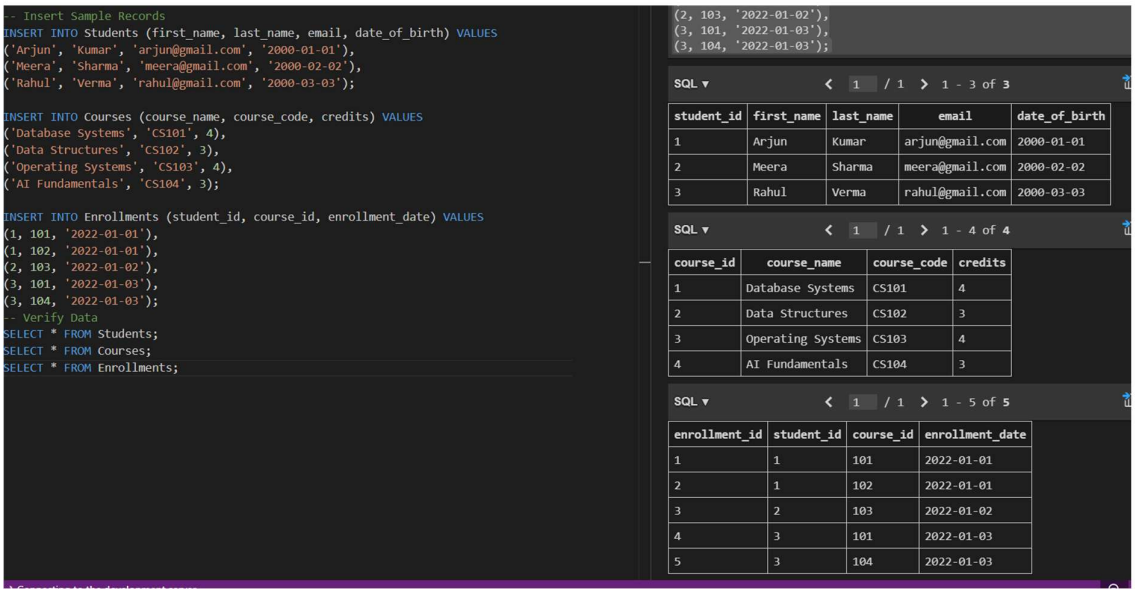
(3, 2, 103),

(4, 3, 101),

(5, 3, 104);

Expected Output:

- Sample records are inserted successfully.
- FROM Students; displays all student records.
- SELECT * FROM Enrollments; displays all enrollments.



```
-- Insert Sample Records
INSERT INTO Students (first_name, last_name, email, date_of_birth) VALUES
('Arjun', 'Kumar', 'arjun@gmail.com', '2000-01-01'),
('Meera', 'Sharma', 'meera@gmail.com', '2000-02-02'),
('Rahul', 'Verma', 'rahul@gmail.com', '2000-03-03');

INSERT INTO Courses (course_name, course_code, credits) VALUES
('Database Systems', 'CS101', 4),
('Data Structures', 'CS102', 3),
('Operating Systems', 'CS103', 4),
('AI Fundamentals', 'CS104', 3);

INSERT INTO Enrollments (student_id, course_id, enrollment_date) VALUES
(1, 101, '2022-01-01'),
(1, 102, '2022-01-01'),
(2, 103, '2022-01-02'),
(3, 101, '2022-01-03'),
(3, 104, '2022-01-03');
-- Verify Data
SELECT * FROM Students;
SELECT * FROM Courses;
SELECT * FROM Enrollments;
```

```
(2, 103, '2022-01-02'),
(3, 101, '2022-01-03'),
(3, 104, '2022-01-03');
```

SQL ▾ 1 / 1 1 - 3 of 3

student_id	first_name	last_name	email	date_of_birth
1	Arjun	Kumar	arjun@gmail.com	2000-01-01
2	Meera	Sharma	meera@gmail.com	2000-02-02
3	Rahul	Verma	rahul@gmail.com	2000-03-03

SQL ▾ 1 / 1 1 - 4 of 4

course_id	course_name	course_code	credits
1	Database Systems	CS101	4
2	Data Structures	CS102	3
3	Operating Systems	CS103	4
4	AI Fundamentals	CS104	3

SQL ▾ 1 / 1 1 - 5 of 5

enrollment_id	student_id	course_id	enrollment_date
1	1	101	2022-01-01
2	1	102	2022-01-01
3	2	103	2022-01-02
4	3	101	2022-01-03
5	3	104	2022-01-03

Task Description- 3: (Perform CRUD Operations)

Instructions:

Use AI to generate SQL queries for basic CRUD operations:

- Retrieve all courses
- Update a student's email
- Delete an enrollment record
- Validate the changes using SELECT statements.

Sample Code:

SELECT * FROM Courses;

UPDATE Students

SET email = 'arjun_new@gmail.com'

WHERE student_id = 1;

DELETE FROM Enrollments

WHERE enrollment_id = 5;

Expected Output:

- Updated email is reflected in Students table.
- Deleted enrollment record is removed successfully.
- Remaining data remains unchanged.

```sql
-- Retrieve all courses
SELECT * FROM Courses;
-- Update a student's email
UPDATE Students
SET email = 'arjun_new@gmail.com'
WHERE student_id = 1;
-- Validate the update
SELECT * FROM Students WHERE student_id = 1;
-- Delete an enrollment record
DELETE FROM Enrollments
WHERE enrollment_id = 5;
-- Validate the deletion
SELECT * FROM Enrollments WHERE enrollment_id = 5;
```

SQL ▾ 1 / 1 1 - 4 of 4

course_id	course_name	course_code	credits
1	Database Systems	CS101	4
2	Data Structures	CS102	3
3	Operating Systems	CS103	4
4	AI Fundamentals	CS104	3

SQL ▾ 1 / 1 1 - 0 of 0

SQL ▾ 1 / 1 1 - 1 of 1

student_id	first_name	last_name	email	date_of_birth
1	Arjun	Kumar	arjun_new@gmail.com	2000-01-01

enrollment_id	student_id	course_id	enrollment_date
1	1	101	2022-01-01
2	1	102	2022-01-01
3	2	103	2022-01-02
4	3	101	2022-01-03
5	3	104	2022-01-03

enrollment_id	student_id	course_id	enrollment_date
1	1	101	2022-01-01
2	1	102	2022-01-01
3	2	103	2022-01-02
4	3	101	2022-01-03

Task Description- 4: (Execute Aggregate Queries)

Instructions:

Use AI tools to write aggregate queries:

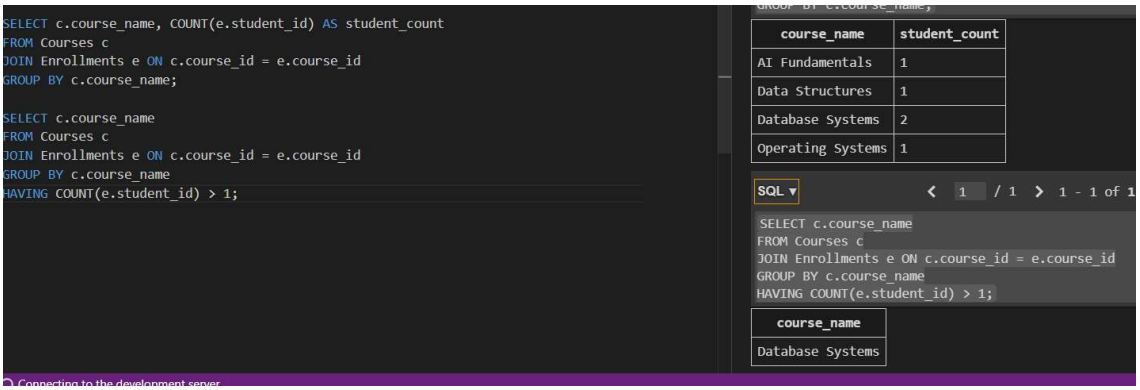
- Count number of students enrolled per course
- Display courses with more than one student enrolled
  Analyze the correctness of results.

Sample Code:

```
SELECT c.course_name, COUNT(e.student_id) AS student_count

FROM Courses c

JOIN Enrollments e ON c.course_id = e.course_id

GROUP BY c.course_name;


SELECT c.course_name

FROM Courses c

JOIN Enrollments e ON c.course_id = e.course_id

GROUP BY c.course_name

HAVING COUNT(e.student_id) > 1;
```

Expected Output:

- Correct count of students per course is displayed.
- Courses with multiple enrollments are listed accurately.



Task Description- 5: (Query Optimization)

Instructions:

- Use AI tools to analyze query performance.
- Rewrite inefficient sub queries using JOINs and test both queries for correctness.

Sample Code:

```
-- Original Query

SELECT * FROM Students

WHERE student_id IN (

   SELECT student_id FROM Enrollments WHERE course_id = 101

);


-- Optimized Query

SELECT s.*

FROM Students s

JOIN Enrollments e ON s.student_id = e.student_id

WHERE e.course_id = 101;
```

Expected Output:

- Both queries return the same set of students.
- Optimized query performs better on large datasets.

This lab allows students to **use AI tools like Copilot, Cursor, or Gemini** to:

- Generate schema definitions

- Auto-complete insert statements
- Write CRUD and aggregate SQL queries
- Optimize queries and ensure proper normalization

```sql
SELECT *
FROM Students
WHERE student_id IN (
    SELECT student_id
    FROM Enrollments
    WHERE course_id = 1
);

SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 1;
```

SQL ▾ 1 / 1 1 - 2 of 2

student_id	first_name	last_name	email	date_of_birth
1	Arjun	Kumar	arjun@gmail.com	2000-01-01
3	Rahul	Verma	rahul@gmail.com	2000-03-03

SQL ▾ 1 / 1 1 - 2 of 2

```sql
SELECT s.*
FROM Students s
JOIN Enrollments e ON s.student_id = e.student_id
WHERE e.course_id = 1;
```

student_id	first_name	last_name	email	date_of_birth
1	Arjun	Kumar	arjun@gmail.com	2000-01-01
3	Rahul	Verma	rahul@gmail.com	2000-03-03